In [1]:
```python
import numpy as np
```

In [2]:
```python
import pandas as pd
```

In [3]:
```python
import matplotlib.pyplot as plt
```

In [5]:
```python
df=pd.read_csv("Downloads/synthetic_diabetes_dataset.csv")
```

In [6]:
```python
df.head()
```

Out[6]:

| | age | BMI | sex | family_history | blood_pressure_status | history_high_blood_sugar | phys |
|---|---|---|---|---|---|---|---|
| **0** | 70-80 | 29.5 | Male | Yes | Normal | Yes | |
| **1** | 60-70 | 26.3 | Male | Yes | Normal | Yes | |
| **2** | 60-70 | 30.2 | Male | No | Prehypertension | No | |
| **3** | 80-90 | 34.6 | Male | No | Normal | No | |
| **4** | 80-90 | 25.8 | Female | Yes | Normal | Yes | |

In [7]:
```python
df.isnull().sum()
```

Out[7]:
```
age                         0
BMI                         0
sex                         0
family_history              0
blood_pressure_status       0
history_high_blood_sugar    0
physical_activity           0
smoking_status              0
diabetes                    0
dtype: int64
```

In [8]:
```python
from sklearn.preprocessing import LabelEncoder
```

In [9]:
```python
le = LabelEncoder()
for col in df.select_dtypes(include=['object']).columns:
    df[col] = le.fit_transform(df[col])
```

In [14]:
```python
X = df.drop("diabetes", axis=1)
```

In [15]:
```python
y = df["diabetes"]
```

In [16]:
```python
print(X.head())
```

```
   age   BMI  sex  family_history  blood_pressure_status  \
0    5  29.5    1               1                      1   
1    4  26.3    1               1                      1   
2    4  30.2    1               0                      2   
3    6  34.6    1               0                      1   
4    6  25.8    0               1                      1   

   history_high_blood_sugar  physical_activity  smoking_status  
0                         1                  0               1  
1                         1                  2               1  
2                         0                  2               2  
3                         0                  2               2  
4                         1                  2               2  
```

In [17]:
```
print(y.head())
```

```
0    1
1    1
2    1
3    0
4    1
Name: diabetes, dtype: int64
```

In [18]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

In [19]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [20]:
```
from sklearn.naive_bayes import GaussianNB
```

In [21]:
```
gaussian = GaussianNB()
```

In [22]:
```
gaussian.fit(X_train, y_train)
```

Out[22]:
```
GaussianNB

GaussianNB()
```

In [27]:
```
Y_pred = gaussian.predict(X_test)
```

In [28]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

In [29]:
```
accuracy = accuracy_score(y_test, Y_pred)
```

In [33]:
```
print("Accuracy:",accuracy)
```

```
Accuracy: 0.85
```

In [30]:
```
precision = precision_score(y_test, Y_pred, average='micro')
```

In [34]:
```
print("Precision",precision)
```

```
Precision 0.85
```

In [31]:
```
recall = recall_score(y_test, Y_pred, average='micro')
```

In [35]:
```
print("Recall",recall)
```

```
Recall 0.85
```

In [37]:
```
from sklearn.metrics import precision_score, confusion_matrix, accuracy_score, reca
```

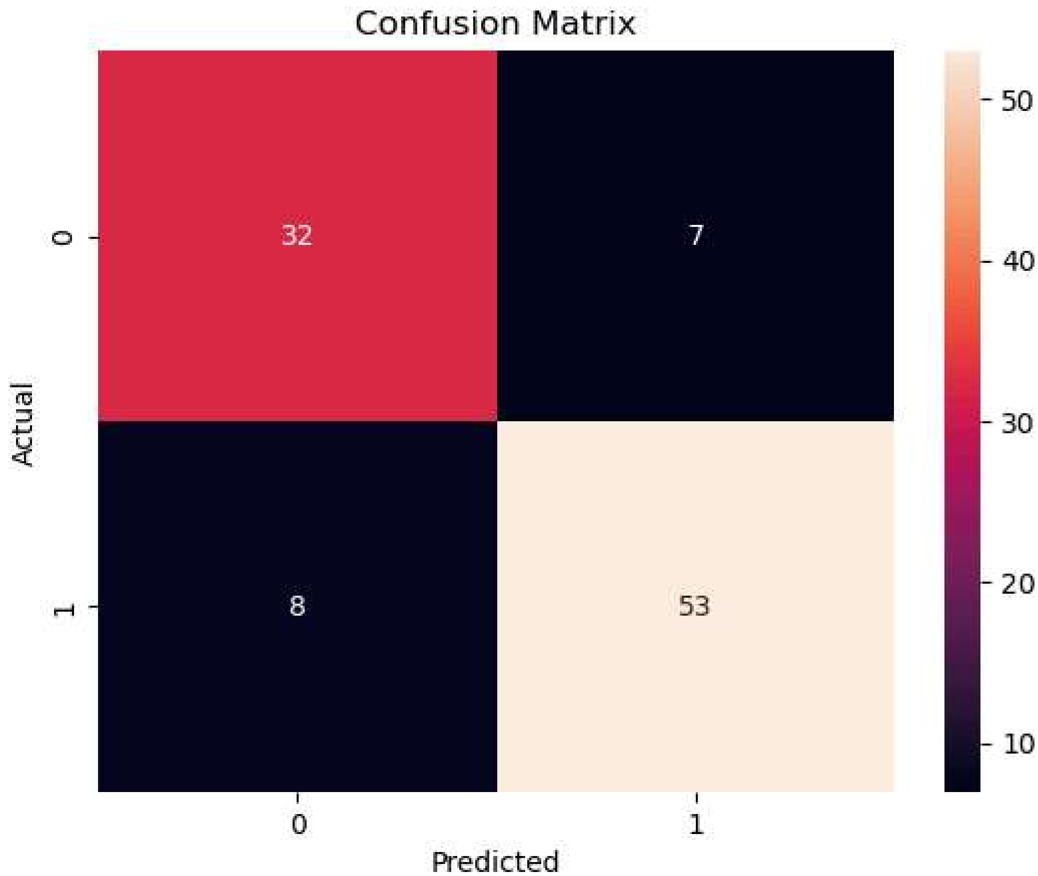
In [38]:
```
cm = confusion_matrix(y_test, Y_pred)

print("Confusion Matrix:\n", cm)
```

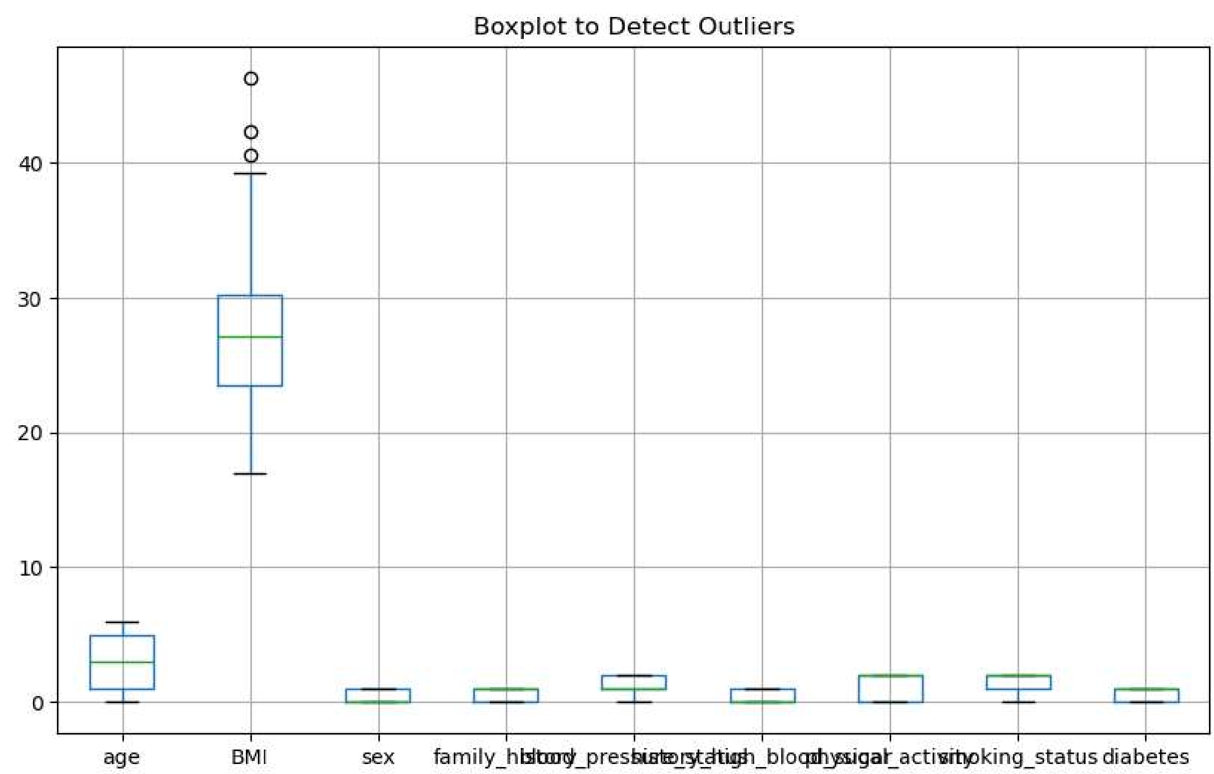
```
Confusion Matrix:
 [[32  7]
 [ 8 53]]
```

In [39]:
```
import seaborn as sns

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




In [40]:
```
df.boxplot(figsize=(10,6))
plt.title("Boxplot to Detect Outliers")
plt.show()
```

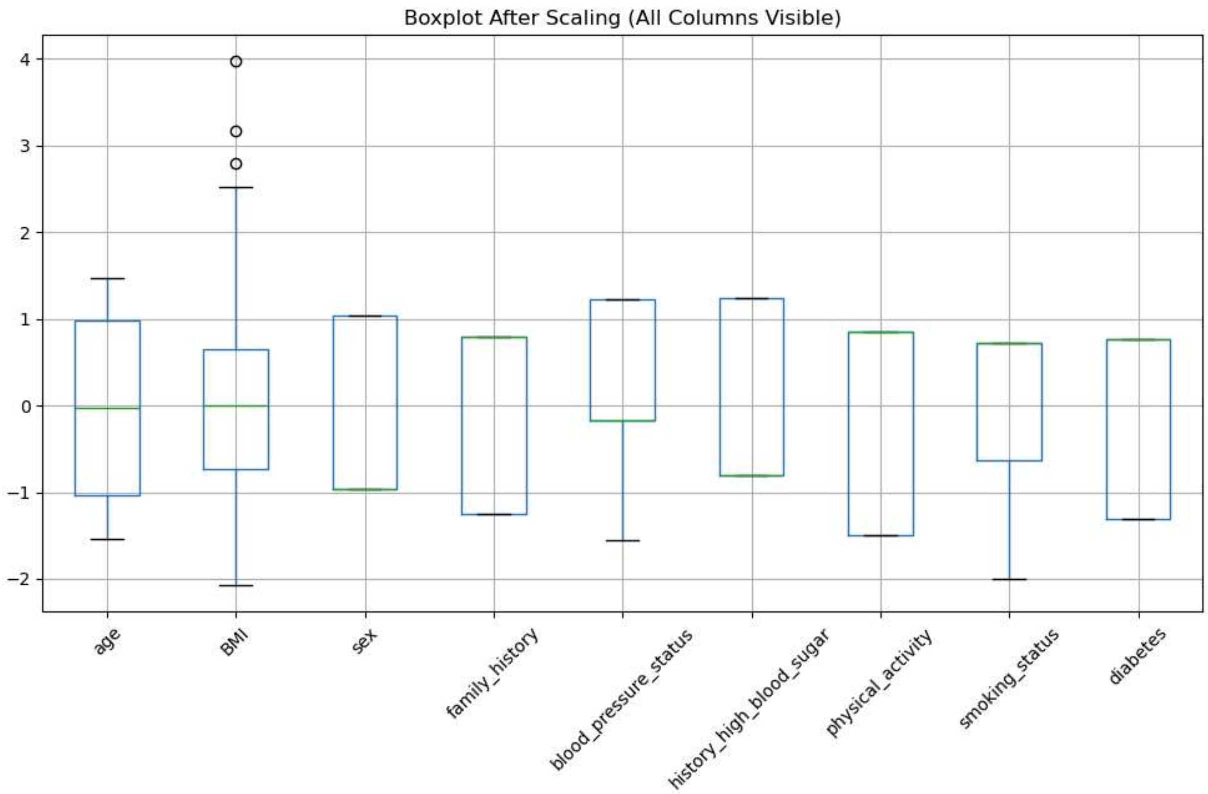


In [56]:
```
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

plt.figure(figsize=(12,6))
scaled_df.boxplot()
plt.xticks(rotation=45)
plt.title("Boxplot After Scaling (All Columns Visible)")
plt.show()
```




In [ ]: